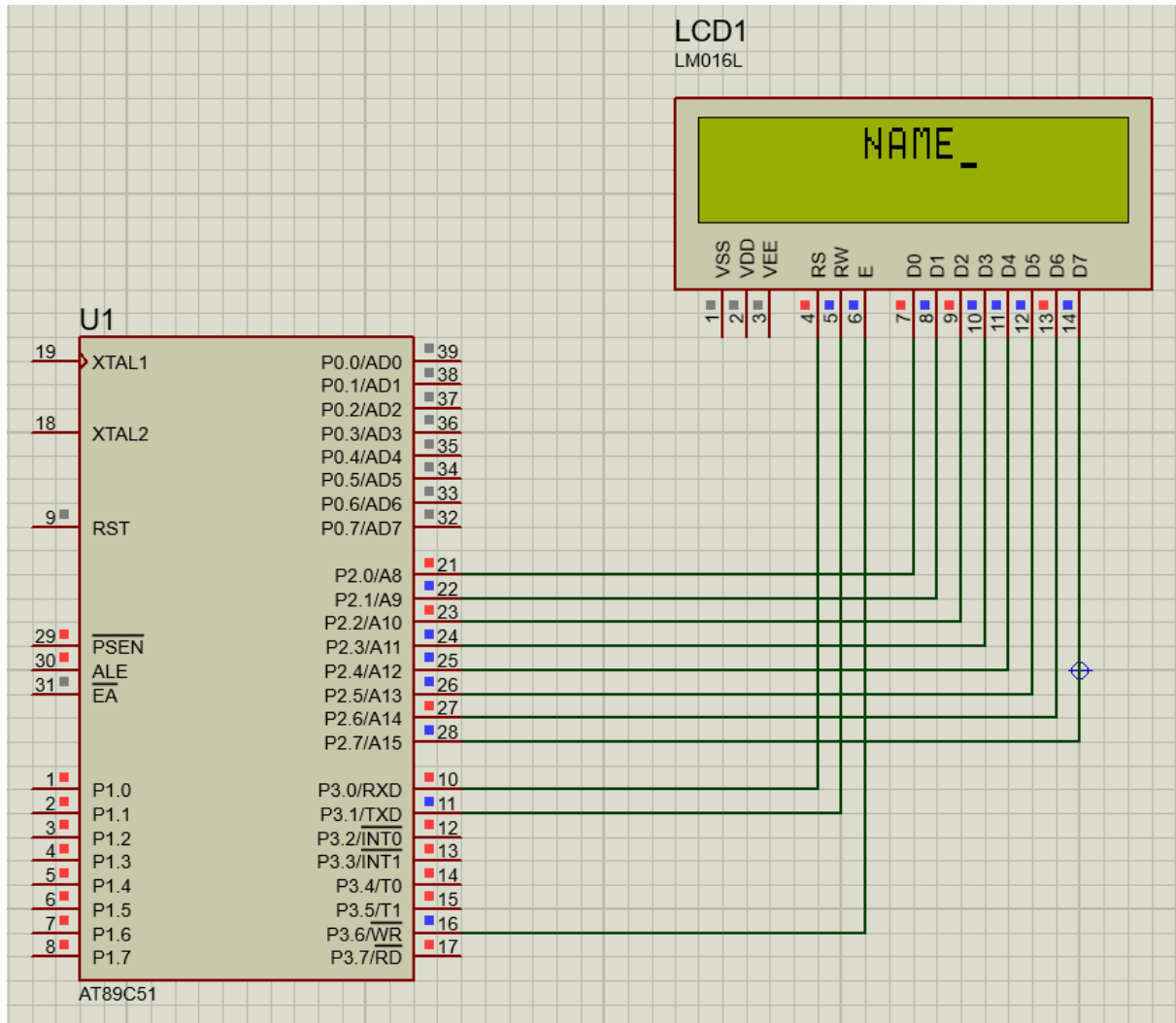


4 16x2 LCD - Print Message

Aim: To write and execute a program to interface an LCD display with the 8051 microcontroller & display a message.



Apparatus Required:

- 8051 microcontroller
- LCD (16×2)
- Resistor / Potentiometer
- Breadboard
- Connecting wires
- Power supply (5V)

- Keil uVision IDE

Theory:

- An LCD is an electronic display module widely used in embedded systems to display text & numeric info.
- It works on the principle of liquid crystals. A 16×2 LCD can display 16 characters in each of its 2 rows.
- The control pins of the LCD include RS (Register Select), EN (Enable) & RW (Read/Write).
- **RS** determines whether the input is a command or data.
- **EN** is used to latch the data into the LCD.
- **RW** selects Read or Write operations.
- The microcontroller communicates with the LCD either in 8-bit mode or 4-bit mode.
- In this experiment, 8-bit mode is used where full data is sent at once, making communication simpler & faster.

Working:

- Commands are sent to the LCD for initialization.
- Data is sent to display characters.
- The Enable pin is toggled (High to Low) to latch data.

LCD Command Table:

Hex Command	Command Name	Description
0x01	Clear Display	Clears the LCD screen
0x02	Return Home	Moves cursor to home/starting position
0x04	Entry Mode Set (Rev)	Moves cursor to left after each character
0x06	Entry Mode Set	Moves cursor to right after each character
0x0C	Display ON	Display ON, cursor OFF
0x0E	Display ON Control	Turns display ON, cursor ON, blinking OFF
0x10	Cursor Shift Left	Moves cursor one position left
0x14	Cursor Shift Right	Moves cursor one position right
0x38	Function Set	Sets 8-bit mode, 2-line display, 5×7 font
0x86	Set DDRAM Address	Moves cursor to 0th position of 1st line

Program Code:

```
#include <reg51.h>

#define cmdport P3
#define dataport P2

// --- LCD Control Pin Definitions ---
sbit rs = cmdport^0; // Register Select: 0 = Command Mode (setup),
1 = Data Mode (text)
sbit rw = cmdport^1; // Read/Write: 0 = Write to LCD, 1 = Read from LCD
sbit en = cmdport^6; // Enable: Needs a quick High-to-Low pulse to lock in the data

// Standard hardware delay
void delay(unsigned int msec)
{
    unsigned int i, j;
    for(i = 0; i < msec; i++)
    {
        for(j = 0; j < 1275; j++); // Approx 1ms delay at 11.0592MHz/12MHz
    }
}

// Send a configuration command (like clear screen, set cursor) to the LCD
void lcdcmd(unsigned char item)
{
    dataport = item;
    rs = 0; // Set to Command Mode
    rw = 0; // Set to Write
    en = 1; // Pull Enable High
    delay(1);
    en = 0; // Pull Enable Low (This falling edge locks the command into the LCD)
}

// Send actual text characters to the LCD
void lcddata(unsigned char item)
{
    dataport = item;
```

```

    rs = 1; // Set to Data Mode (Text)
    rw = 0; // Set to Write
    en = 1; // Pull Enable High
    delay(1);
    en = 0; // Pull Enable Low (This falling edge prints the character)
}

void main()
{
    // LCD Initialization Sequence
    lcdcmd(0x38); delay(100); // Wakes up LCD: 8-bit mode, 2 lines, 5x7 font matrix
    lcdcmd(0x0E); delay(100); // Display ON, Cursor ON (Blinking)
    lcdcmd(0x01); delay(100); // Clear Display Screen completely
    lcdcmd(0x06); delay(100); // Entry Mode: Auto-increment cursor to the right
    lcdcmd(0x86); delay(100); // Move cursor to Line 1, Position 6

    // Displaying Message: "NAME"
    lcddata('N'); delay(100);
    lcddata('A'); delay(100);
    lcddata('M'); delay(100);
    lcddata('E'); delay(100);

    // Infinite loop safely traps the microcontroller so it doesn't restart and flicker
    while(1);
}

```

Result: The LCD successfully displayed the message.

Precautions:

- Check LCD connections properly.
- Ensure proper power supply.
- Avoid wrong pin connections.